임베디드 시스템 프로젝트 중간보고서

소속대학	공과대학	소속 학과	미래자동차공학과
학번	2019048440	성명	최윤석

▶임베디드 시스템 실습수업 내용(Road Following, Object Detection 등) 구현 방식
-Task 1. 신호등 인식 기반 정지/진행 제어
목표: 차량이 빨간 불(red_light)을 인식했을 때 정지하고, 빨간불이 꺼지면 자동으로 다시 전진
학습: CVAT로 라벨링한 신호등(red, green) 데이터 셋 학습
Inference: 실시간 YOLO 추론 기반으로 신호등 색상 판단
구한 방법: 빨간 불 시 base.stop() 명령으로 정지 → 빨간 불이 아닌 경우 전진
결과: 실시간 제어 가능, 다만 자연광 등의 외부 환경 변화에 따라 인식률이 다소 차이 발생

Task 1

- 신호등 빨간 불(red_light)를 인식하면 정지
- red_light이 아닌 경우 전진

```
if red_lights:
    red_lights.sort(reverse=True, key=lambda x: x[0])
    nearest_area, _ = red_lights[0]
    if nearest_area >= 450:
        print("[TRAFFIC] RED LIGHT → STOP")
        return 0.0, 0.0
```

-Task 2. SLOW, STOP 표지판 기반 속도 제어
*목표: 도로상의 SLOW 또는 STOP 표지판 인식 시 차량의 속도 변화 유도
*학습: 표지판 종류별로 YOLO 모델 학습 (SLOW, STOP 라벨링 포함)
*구현 방법:인식 거리 내 SLOW 표지판 감지 시 5초간 감속, STOP 표지판 감지 시 3초간 정지 후 자동 복귀
*결과: 명확하게 표지판을 인식할 경우 정상 작동, 다만 주변 배경이 복잡한 경우 잘못 탐지 가능성 존재

Task 2

- 일정 거리 내에 SLOW, STOP 표지판을 인식
- STOP에는 3초 동안 멈춘 후 다시 전진

```
if stop_mode:
    if now - stop_start_time < STOP_DURATION:
        print(f"[STATE] STOP MODE ({now - stop_start_time:.1f}/{STOP_DURATION}s)")
        return 0.0, 0.0
    else:
        stop_mode = False
        print("[STATE] STOP MODE → NORMAL")
```

Task 2

- 일정 거리 내에 SLOW, STOP 표지판을 인식
- SLOW에는 5초 동안 느리게 간 후 다시 전진

```
if slow_mode:
    if now - slow_start_time < SLOW_DURATION:
        print(f"[STATE] SLOW MODE ({now - slow_start_time:.1f}/{SLOW_DURATION}s)")
        return SLOW_SIGN_SPEED, steering
    else:
        slow_mode = False
        print("[STATE] SLOW MODE → NORMAL")
```

-Task 3: Vehicle 회피 제어
*목표: 3단계로 나눠서 회피 실행(Avoiding->Straight->Recovery)
*학습: CVAT로 라벨링한 차량(Vehicle) 데이터 셋 학습
*구현 방법:
1) Avoiding:Wave Rover와 Vehicle 사이의 적절한 거리에 도달했을 때 차량을 인식하고, Wave Rover를 틀어서 회피 방향을 결정하도록 알고리즘 구성

Task 3

- 차량을 인식 후 회피 기동 실행

- 3단계로 나눠서 회피 실행

1) Avoiding

2) Straight

3) Recovery

```
if avoidance_state == 'avoiding':
    if avoidance_timer < AVOID_DURATION:
        steering = 0.5 * avoiding_dir
        throttle = 0.25
        print("avoid duration")
    else:
        avoidance_state = 'straight'
        avoidance_timer = 0
```

2) Straight: 차량을 피할 수 있는 방향으로 회전 후 직진하는 과정. 여기에서도 2가지의 서브 단계(Two Sub-Steps)로 구현

<1> straight-align: 사다리꼴의 꼭짓점으로 정의. 주행 도로에서 크게 벗어나지 않도록 Avoiding 단계의 반대 방향으로 steering 하도록 알고리즘 구성

<2> straight: 차량을 직진 주행시켜 정지된 Vehicle과의 거리가 멀어지도록 알고리즘 구성

Task 3

- 차량을 인식 후 회피 기동 실행
  - 3단계로 나눠서 회피 실행
    1) Avoiding
    2) Straight
    3) Recovery

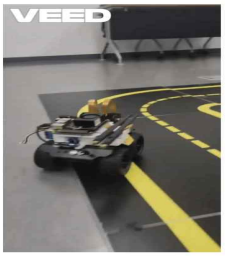
```
elif avoidance_state == 'straight':
    if avoidance_timer < STRAIGHT_ALIGN_DURATION:
        steering = 0.2 * avoiding_dir
        throttle = 0.3
        print("straight align duration")
    elif avoidance_timer < STRAIGHT_DURATION:
        steering = 0.0
        throttle = 0.25
        print("straight duration")
    else:
        avoidance_state = 'recovery'
        avoidance_timer = 0
```

3) Recovery: 회피 후 본래 주행 도로로 돌아가기 위한 과정으로서, 복귀 시 차량의 중앙선을 따라가도록 알고리즘 구성. straight와 비슷하게 2가지의 서브 단계 구현.
<1> recovery: Wave Rover를 도로 밖에서 본래 주행 도로를 향해 steering 하도록 알고리즘 구성
<2> recovery align: wave rover를 본래 주행 도로의 중앙선을 따라가도록 알고리즘 구성

Task 3

- 차량을 인식 후 회피 기동 실행
  - 3단계로 나눠서 회피 실행
    1) Avoiding
    2) Straight
    3) Recovery

```
elif avoidance_state == 'recovery':
    if avoidance_timer < RECOVERY_DURATION:
        steering = 0.5 * recovery_dir
        throttle = 0.25
        print("recovery duration")
    elif avoidance_timer < RECOVERY_DURATION + RECOVERY_ALIGN_DURATION:
        steering = 0.4 * recovery_dir
        throttle = 0.4
        print("recovery align duration")
    else:
        avoidance_state = 'center'
        avoidance_timer = 0
        avoidance_ignore_counter = AVOID_IGNORE_DURATION
```



*결과: 시연 연습 중에 어느 정도 성공도가 높았지만, 실제 시연에서는 성공도가 좀 낮은 편이었음.

-Task 4: 신호등과 표지판 기반 교차로 주행

*목표: 차량이 주행 중 가장 가까운 신호등의 red_light 상태를 우선 인식하여 정지하고, 이후 신호등이 꺼지면 표지판(Left, Right, Straight)과 일정 거리 내 교차로(Intersection)를 동시에 인식하여 주행 방향을 결정

*학습: CVAT에서 신호등, 진행 표지판, 교차로 각각에 대해 개별적으로 라벨링 수행 (라벨 종류: red_light, green_light, Left, Right, Straight, Intersection) 및 학습된 YOLO 모델을 기반으로 3종 객체 동시 인식 가능하도록 설계
*구현 방법:
1)신호등(red_light)이 감지되면 차량 정지
2)red_light 꺼짐 감지 시 진행 표지판 인식 및 일정 거리 내 Intersection 감지
3)표지판에 따라 좌회전, 우회전, 직진 중 한 경로 선택
*결과:
실제 주행 환경에서 다중 객체 동시 인식 기반 제어 성공. 단, 밝기나 바닥 반사 등 외부 요인에 따라 교차로 인식 정확도 편차 발생. 특히 표지판과 교차로 간 간격이 짧을 경우 경로 판단에 지연 가능성 존재

-Dataset 제작 및 학습 정보
*총 27,000 프레임 수집 → CVAT를 이용한 라벨링
*Task별 라벨: 신호등(red, green), 도로 표지(SLOW, STOP, LEFT, RIGHT)
*학습: YOLO 기반 객체 검출 (각 Task당 약 150 Epoch 학습, 총 소요 시간 약 35시간)
*학습 환경: Jetson Orin Nano + PyTorch + YOLOv8
*다량 프레임 수집 및 활용 목적: 실제 주행 환경에서는 다양한 상황이 발생할 수 있어 이러한 상황들을 미리 고려하여 여러 조건을 설정.

*데이터 셋 제작 방법: 실시간으로 찍은 사진들을 영상으로 제작. 그 후, CVAT 사이트에서 라벨링 후 학습 과정은 개인 노트북으로 진행 (GPU 사용에 용이함)

```
sudo apt update && sudo apt install ffmpeg                                    // ffmpeg 설치

ffmpeg -framerate 30 -i frames/frame_%06d.jpg -c:v libx264 -pix_fmt yuv420p output_video.mp4   // ffmpeg로 동영상 제작
```

-전체 경로 및 주행 구조
중앙선을 기준으로 경로 설계(특히, Alexnet상에서 주행 실험 중에 찍힌 프레임 중에 실선 하나만 보이는 프레임이 있다면 최대한 중앙선으로 가도록 점을 찍어가면서 라벨링 작업을 진행함) 최대한 경로 이탈을 방지하기 위해 PID 기반 조향 제어 적용. Task 수행 전/후 경로 안정성 확인 실험실시

```
# ================== PID 파라미터 ==================
Kp, Kd, Ki = 1.0, 0.15, 0.1
turn_threshold = 0.7
integral_threshold = 0.2
integral_min, integral_max = -0.4 / Ki, 0.4 / Ki
cruise_speed, slow_speed = 0.45, 0.35
```

```
if abs(err) > integral_threshold or prev_err * err < 0:
    integral = 0.0
else:
    integral = np.clip(integral + err * dt, integral_min, integral_max)

steering = Kp * err + Kd * (err - prev_err) / dt + Ki * integral
prev_err = err

throttle = cruise_speed if abs(steering) < turn_threshold else slow_speed

throttle, steering = handle_task_logic(frame_bgr, throttle, steering)
```

▶기타 고려 사항
1) 자연광 유무에 따른 주행 성공도 차이가 큼(도로 라인을 인식하는 데 어려움을 겪음). 따라서, 최대한 자연광의 개입이 없는 상태에서 경로 주행 실험을 수행함
2) 라벨링을 한 적도 없는 관계없는 객체를 인식하는 경우가 있음. 예를 들어,

	사람이나 의자 바퀴, 소화기 등이 Vehicle로 인식되는 오류가 발생함. 다른 객체들의 배치나 조명을 적절히 조정해서 차량이 최대한 안정적으로 주행할 수 있도록 설계가 필요하다는 것을 느낌. 3) 분명 같은 주행 알고리즘을 활용했는데도 불구하고 Wave Rover 충전 상태에 따라 주행 성공가 제각기였음. 즉, Wave Rover 충전이 많이 됐을 때에는 throttle이나 steering을 조금만 늘려도 output이 큰 폭으로 증가하여 회피 단계 전반에 걸쳐 steering이 너무 커서 본래 주행 경로의 중앙선을 잘 못 찾아가는 문제가 생김. 그래서 Wave Rover 충전 상태를 10.3~10.4V 사이로 설정하고 그 상태에서 속도와 조향 정도를 조절함. ▶아쉬웠던 점 1) 첫 번째 시도 주행 중에는 전체적으로 Task 4를 제외한 나머지는 완벽히 수행하였지만, 주행 중에 좌우로 진동하면서 비록 경로는 벗어나지 않았지만 다소 불안정하게 주행하는 경향성을 보임. 여기서 본인은 PID 제어 상수 중 적분 계수(Ki)의 크기를 조금 감소시켜야겠다는 생각이 들어 0.1에서 0.095로 줄였더니 두 번째 시도 주행부터는 전에 비해 안정적인 주행을 보여줌. 2) 모든 Vehicle에 대해 라벨링을 수행하지 못 함. 우리 조가 가지고 있는 Vehicle로만 라벨링을 진행하였는데, 다른 조의 Vehicle 또한 라벨링을 활용하여 다양한 상황에 더 대비했어야 했지만 그렇게 하지는 못함. 그 결과 두 번째와 세 번째 주행 시도에서 Task 3(정지 차량 회피)을 수행하는 것에 실패했다고 봄. 3) 첫 번째 주행 시도와 두 번째 주행 시도에서 직진 표지판이 등장하였고, 직진 표지판 자체는 잘 인식했지만 실행 단계에서 직진이 아닌 회전을 하는 문제가 발생함. ▶향후 개선 방향 및 기말 프로젝트 목표 이번 중간 프로젝트에서는 신호등, 표지판, 차량 등이 정적인(Static) 상태에 있는 시나리오를 기반으로 Wave Rover의 반응을 설계하고 구현하였습니다. 특히, 신호등 정지 및 표지판 기반 제어, 정지된 차량 회피와 같은 Task를 수행하며 기본적인 인지-판단-제어 흐름을 구현해보는 데에 중점을 두었습니다. 하지만 시연 중 정지 차량 회피(Task 3)의 경우, 첫 번째 주행을 제외하고는 정상적으로 수행하지 못하였으며, 이는 실제 주행 중 발생할 수 있는 다양한 상황에 대한 대응력이 부족했음을 시사합니다. 따라서 기말 프로젝트(Your Own Problem)에서는 보다 동적인(Dynamic) 환경에 대한 대응을 목표로 설정하였습니다. 예를 들어, 차량(Vehicle)이 갑자기 급정지하는 상황에서는 Wave Rover가 일정 거리 이내에서 자동 정지하도록, 차량이 급후진하는 경우에는 Wave Rover가 회피 기동을 수행하도록 알고리즘을 구성할 예정입니다. 이러한 시나리오는 실제 주행 환경에서 충분히 발생이 가능한 상황이며, Wave Rover가 주변 객체의 상태 변화에 능동적으로 반응하도록 개선하는 것이 이번 기말 프로젝트의 핵심 목표입니다. 이를 통해 중간 프로젝트에서 부족했던 회피 제어의 완성도를 높이고, 정적 상황 대응을 넘어서 동적 위험 회피 기능까지 포함한 자율 주행 제어 시스템을 구현하고자 합니다.